ЛАБОРАТОРНАЯ РАБОТА №1
Безопасная разработка и анализ приложения по ГОСТ

Теория → пошаговая практика → проверка результата
2 академических часа · учебный проект vulnerable-app

Выполняем только на локальной учебной копии. Реальные секреты и чужие системы не используем.

1. Что вы изучите

Научиться смотреть на приложение как специалист по ИБ: выделять данные, границы доверия, угрозы и формулировать требования, которые можно проверить.

• Поймёте, что такое безопасный SDLC.

• Составите схему frontend → backend → БД.

• Опишете активы и минимум пять угроз.

• Сформулируете проверяемые требования безопасности.

Опыт программирования не требуется: команды и фрагменты кода можно копировать. После каждого шага объясняется, что должно получиться и как проверить результат.

2. Теория простыми словами

Что такое безопасная разработка

Это не отдельная проверка перед сдачей проекта. Требования безопасности появляются в начале, учитываются при проектировании, проверяются в коде и после выпуска. Исправлять причину дефекта важнее, чем отключить предупреждение сканера.

Что регулирует ГОСТ

ГОСТ Р 56939-2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования» описывает работы по созданию безопасного ПО и устранению недостатков. Для статического анализа дополнительно применяется ГОСТ Р 71207-2024 «Защита информации. Разработка безопасного программного обеспечения. Статический анализ программного обеспечения. Общие требования». Наши лаборатории показывают отдельные практики, но не подтверждают соответствие всей организации ГОСТу.

Frontend, backend и база данных

Frontend работает в браузере: HTML, CSS и JavaScript. Backend обрабатывает запросы и решает, что можно делать пользователю. База хранит пользователей, записи и комментарии. Проверка только в браузере недостаточна: нарушитель может отправить запрос напрямую backend.

Активы и границы доверия

Актив — то, что нужно защищать. В нашем проекте это пароли, email, закрытые заметки, токен и записи БД. Граница доверия — место, где данные переходят между компонентами. Вход пользователя всегда считается недоверенным, пока backend не проверил его.

Угроза, уязвимость и риск

Уязвимость — конкретный дефект, например SQL injection. Угроза — возможное действие нарушителя. Риск зависит от вероятности и последствий: какие данные откроются, доступен ли маршрут, нужны ли права. Одна и та же находка может иметь разный риск в разных системах.

Хорошее требование безопасности

Плохо: «сделать профиль безопасным». Хорошо: «при запросе чужого профиля backend возвращает 403, а без сессии — 401». Такое требование можно проверить тестом.

3. Перед началом

• Распакуйте практикум и откройте терминал в каталоге vulnerable-app.

• Нужны Python 3.11+, браузер и терминал. Для ZAP и альтернативных запусков — Docker Desktop.

• Команды показаны для Git Bash. Для PowerShell используйте .venv\Scripts\Activate.ps1 вместо source.

• Создайте папку reports для результатов. Не публикуйте отчёты, пока не проверили их содержимое.

• Сохраняйте исходную копию проекта, чтобы сравнивать состояние до и после исправлений.

4. Практика по шагам

Шаг 1. Запустить учебное приложение

Зачем: Нужно убедиться, что проект работает до анализа. Все команды выполняются в каталоге vulnerable-app.

Что сделать:

1. Откройте терминал в каталоге vulnerable-app.

2. Создайте виртуальное окружение — отдельный набор Python-пакетов для проекта.

3. Активируйте окружение и установите зависимости.

4. Запустите backend. Терминал после этого не закрывайте.

Команда или код:

```
python -m venv .venv
# Git Bash:
source .venv/Scripts/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt -r requirements-dev.txt
python -m backend.app
```

Ожидаемый результат: В терминале появится адрес http://127.0.0.1:5000. Он открывается только на вашем компьютере.

Если не получилось:

• Если порт 5000 занят, завершите старый процесс или запустите Flask на другом порту.

• Если окружение не активируется в PowerShell, используйте Git Bash.

Шаг 2. Проверить приложение через браузер и тесты

Зачем: Сначала нужно понять обычное поведение системы. Тесты показывают, что учебные сценарии воспроизводятся.

Что сделать:

1. Откройте http://127.0.0.1:5000.

2. Перейдите на /login и войдите как student/student123.

3. Откройте /search, /comments и /profile?id=2.

4. Во втором терминале выполните pytest -q.

Команда или код:

```
pytest -q
```

Пример и объяснение: Пример результата: .... [100%] 4 passed in 0.42s

Ожидаемый результат: Главная страница отображает материалы, вход работает, pytest показывает 4 passed.

Шаг 3. Нарисовать архитектуру

Зачем: Схема помогает увидеть, где именно возникают риски. Не нужно знать весь код.

Что сделать:

1. Откройте backend/app.py и найдите функции login, search, comments и profile.

2. Откройте backend/database.py и найдите users, posts и comments.

3. Нарисуйте Browser → Flask backend → SQLite/PostgreSQL.

4. Подпишите входные данные: логин, пароль, q, comment и id.

5. Отдельно отметьте Docker Compose и порт БД 5432.

Пример и объяснение: Пример подписи на схеме: «id приходит из адресной строки в /profile. Backend обязан проверить, можно ли текущему пользователю видеть этот id».

Ожидаемый результат: Получилась одна страница со стрелками, компонентами и входными данными.

Шаг 4. Составить реестр активов

Зачем: Нельзя защитить систему, если непонятно, какие данные важны.

Что сделать:

1. Создайте таблицу с колонками: актив, где хранится, кто владелец, последствия раскрытия, последствия изменения.

2. Добавьте пароли, email, secret_note, токен, комментарии и записи БД.

3. Для каждого актива оцените влияние словами: низкое, среднее или высокое. Объясните оценку.

Пример и объяснение: secret_note хранится в users. Раскрытие через чужой /profile показывает личную информацию пользователя. Влияние — высокое для владельца записи.

Ожидаемый результат: В таблице не менее шести активов. Есть не только названия, но и последствия.

Шаг 5. Описать угрозы

Зачем: Теперь свяжем маршруты с возможными действиями нарушителя.

Что сделать:

1. Для /login опишите попытку войти без пароля и последствия SQL injection.

2. Для /search опишите ввод HTML/JavaScript и возможный XSS.

3. Для /comments опишите сохранение вредоносной разметки.

4. Для /profile опишите подбор чужого id — IDOR.

5. Для config.py опишите публикацию секрета.

6. Для каждой угрозы укажите актив и способ проверки.

Пример и объяснение: Маршрут: /profile?id=1. Действие: студент меняет id на 1. Результат: видит email и заметку admin. Метод: ручной тест контроля доступа.

Ожидаемый результат: Получилось минимум пять угроз с маршрутом, действием, активом и способом проверки.

Шаг 6. Сформулировать требования и план проверок

Зачем: Требования связывают анализ с исправлениями в следующих лабораторных.

Что сделать:

1. Запишите не менее восьми требований: SQL, XSS, контроль доступа, пароли, секреты, заголовки, debug и порт БД.

2. Для каждого укажите способ проверки: тест, SAST, Gitleaks, Trivy, ZAP или ручная проверка.

3. Составьте таблицу: этап SDLC, действие, результат, ответственный, критерий завершения.

4. Сохраните первичный реестр дефектов без исправлений — он пригодится для сравнения.

Пример и объяснение: Требование: все SQL-запросы с пользовательскими значениями используют параметры. Проверка: Semgrep + попытка SQL injection + повторный pytest.

Ожидаемый результат: Каждое требование можно проверить. Например, «значение q отображается как текст» проверяется повторным запросом и просмотром HTML.

5. Что сдать в отчёте

• Схема архитектуры с границами доверия.

• Реестр не менее шести активов.

• Таблица не менее пяти угроз.

• Восемь проверяемых требований.

• Таблица этапов SDLC и первичный реестр дефектов.

В конце добавьте краткий вывод: какой дефект был найден, какая причина устранена, что проверено повторно и какие риски остались.

6. Проверь себя

1. Почему нельзя считать безопасность только задачей сканера?

2. Чем угроза отличается от уязвимости?

3. Почему id в адресной строке нельзя считать доказательством права доступа?

4. Как выглядит проверяемое требование безопасности?

5. Что фиксируют для воспроизводимого анализа?

7. Критерии оценивания

Критерий	Баллы
Архитектура и активы	25
Угрозы	25
Требования	25
План проверок и отчёт	25

Максимум — 100 баллов. За использование чужой системы без разрешения или публикацию настоящих секретов работа не принимается. Честно описанный отрицательный результат и ограничения инструмента являются нормальной частью анализа.

8. Источники

• ГОСТ Р 56939-2024: https://www.gostinfo.ru/catalog/Details/?id=8094263

• ГОСТ Р 71207-2024: https://protect.gost.ru/gost/details/9dd6f064-6014-4416-a3ca-ef72ce0a6c3a

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

• NIST SSDF: https://csrc.nist.gov/Projects/ssdf